

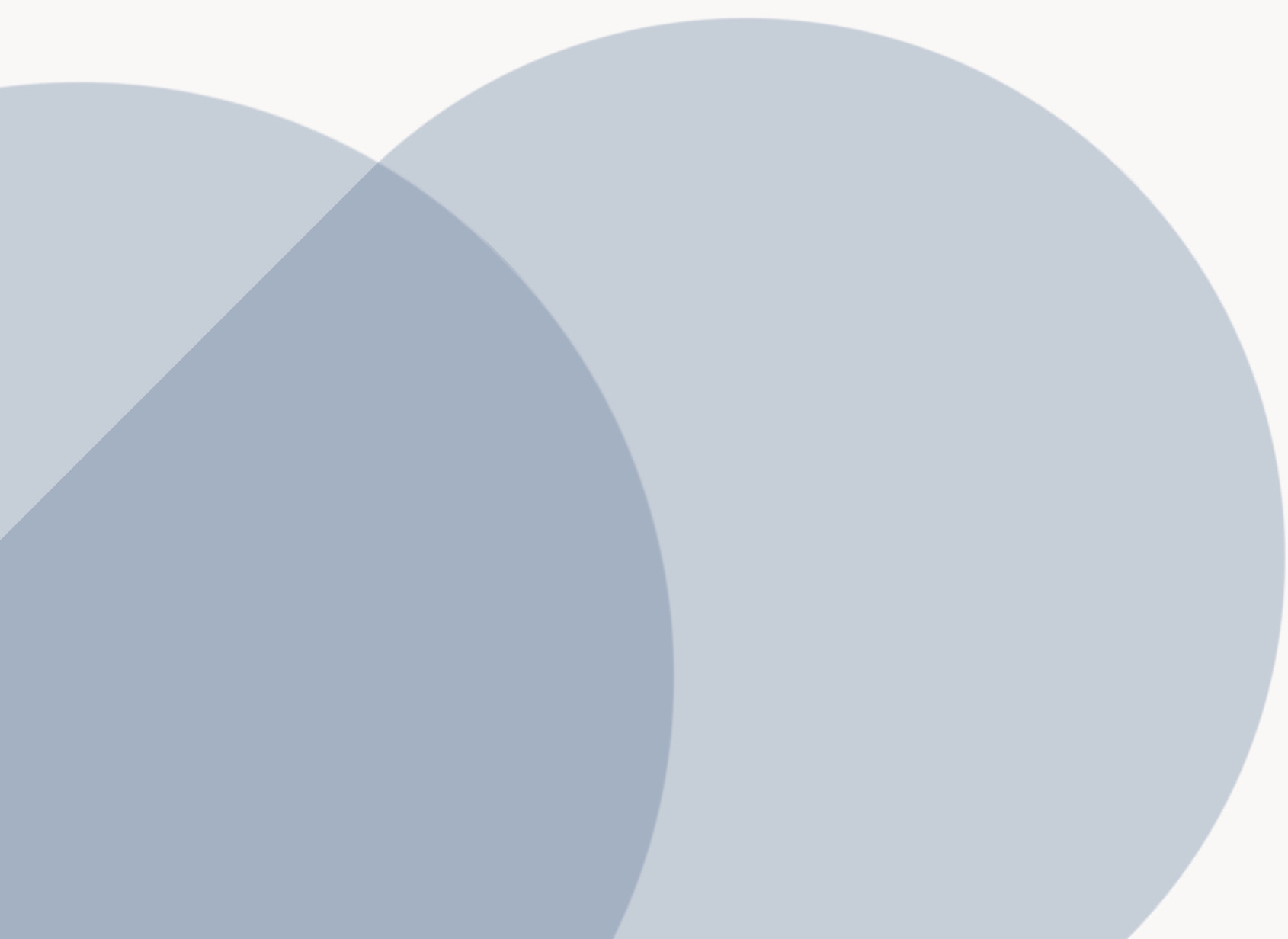
# PORTFOLIO

정합성과 운영 안정성을 함께 고려하는 백엔드 개발자 **옥정현**입니다.

**CONTACT**

devoks@naver.com

# INDEX



---

**01** 자기소개

---

**02** 개인 강점역량

---

**03** 프로젝트 소개

---

# 정합성과 운영 안정성을 함께 고려하는 백엔드 개발자 **옥정현**입니다.

**2025년** UniHub 프로젝트에서 수강 신청 로직과 동시성 제어 구조를 직접 설계하며 개발에 매력을 느꼈고,  
많은 요청이 들어와도 **정확한 처리와 안정적인 운영 구조**를 만드는 백엔드 개발자를 목표로 하게 되었습니다.

**최근에는** 연근 마켓 프로젝트에서 **Elasticsearch** 기반 검색 성능 개선과 **Sentry-Slack** 연동을 통한  
**장애 대응 환경 구축**을 경험하며, 서비스 품질은 기능 구현뿐 아니라 운영 과정에서 함께 완성된다는 점을 배웠습니다.

**현재는** 개인 프로젝트를 진행하며 운영 안정성과 구조 설계 역량을 키우고 있고,  
개인 블로그에 백엔드 기술 흐름과 문제 해결 사례를 꾸준히 정리하며 **100편 이상의 개발 기록**을 남기고 있습니다.



## Education

- 프로그래머스 백엔드 교육과정에서 800여 시간 동안 **Java, Spring Boot** 기반 웹 백엔드 개발 학습
- REST API 설계, 예외 처리, 인증/인가 구조 이해 및 구현 실습
- AWS EC2 배포, GitHub Actions 기반 CI/CD 자동화 파이프라인 실습
- Git, GitHub 활용 및 **프론트엔드 실무자와의 REST API 협업 경험**
- 심화 교육과정을 통해 **MSA, Kafka, Elasticsearch** 기반 아키텍처 설계 학습

## Skills

- **Java, Spring Boot, JPA, Spring Security** 기반 REST API 구현 경험
- **MySQL, Redis, Elasticsearch** 활용 데이터 처리 및 검색 성능 개선 경험
- **Redis** 기반 동시성 제어, 분산락, 비동기 처리 구조 설계 경험
- **AWS EC2, Docker, GitHub Actions** 기반 배포 및 자동화 경험
- **Swagger UI, Sentry-Slack** 기반 문서화 및 모니터링 구축 경험



피어리뷰 ( 팀원들의 긍정적인 평가 )

[피어리뷰] 최종프로젝트

|       |                 |
|-------|-----------------|
| 이름    | 옥정현             |
| 리뷰 인원 | 4               |
| 과정명   | 백엔드 단기심화 1기 1회차 |
| 리뷰 기간 | 2025.12.19      |

5. 좋은 점/칭찬할 점이 있나요 ?

문서화를 꼼꼼하게 잘하십니다!

잠깐 자기소개할 때 개발속도가 느리다고 하셨던거 같은데 전혀 아닌거 같아요.. 빠르세요..

1. 문서나 회의 같은 것들 정리를 너무 잘해서 나도 저렇게 정리를 잘하고 싶다고 생각했습니다. 이제까지 현업이나 프로젝트를 진행했을 때 이보다 더 잘 정리한 사람을 보지 못했습니다. 이 부분은 정말 본인의 강점이니 이 부분을 취업준비하실 때 어필하신다면 도움이 될거라고 생각합니다.

2. 질문이나 도움 요청에 적극적으로 응답해주셔서 좋았습니다. 협업하고 싶은 개발자상입니다

3. 회의나 소통에 적극적이셔서 좋았습니다.

모르는 부분을 질문하면 개인 채팅으로 화면공유로 알려주시거나, 내용을 순서대로 정리해서 자세히 알려주셨습니다. 같은 질문을 해도 짜증이 아니라 다시 차분히 알려주셨고, 그 외에도 pr 리뷰할 때 등 코드를 보면서 이야기 할 때 자세한 설명이 너무 좋습니다. 정현님께서는 말이 많았다며 죄송하다고 그러셨지만 죄송하지 않으셔도 됩니다. 좋은 시간이었고 유익했습니다.

7. 고맙거나 감사한 점이 있나요 ?

개발하면서 정현님 코드 많이 참고했습니다 덕분에 수월하게 개발할 수 있었습니다

PR올린 부분을 설명해주실때도 자세히 설명해주셔서 작업한 내용을 보기 편했습니다

서기를 시키지 않아도 나서서 정리해주시고 심지어 자세히 잘 정리해주셔서 감사했습니다. 뿐만 아니라 프로젝트 하는 모든 순간이 감사했습니다.

PR 리뷰 ( 구체적인 피드백 )

okjunghyeon on Dec 3, 2025

Member

...

외부 PG사 요청(I/O 작업)이 포함된 로직에서 @transactional을 제외하신 점이 매우 좋습니다. 덕분에 네트워크 지연이나 예외 발생 시에도 불필요하게 DB 커넥션을 점유하는 문제를 방지할 수 있겠네요.

또한 deposit 조회 시 엔티티가 아닌 DTO를 사용함으로써, 트랜잭션 범위 밖에서 발생할 수 있는 Lazy Loading 이슈나 영속성 컨텍스트 관련 문제를 사전에 차단하신 점도 좋은 것 같습니다!

👍

shark-coding on Dec 3, 2025

Member

Author

...

좋은 리뷰 감사합니다!

Deposit은 Payment와 동일한 도메인 모듈 내에서 동작하는 구조이므로 Deposit 조회 시 DTO를 직접 사용하기보다는 도메인 객체를 기준으로 처리하는 방식으로 개선하겠습니다.

API 반환 단에서만 DTO 변환이 일어나도록 리팩토링해 레이어 간 책임이 명확히 분리되도록 개선해보겠습니다!

👍 🍷 1

okjunghyeon on Dec 9, 2025 • edited

Member

...

저도 elasticSearch 구현을 하면서 createdAt 타입 불일치로 동기화가 안되는 문제를 겪었습니다.

```
@Field(
    name = "created_at",
    type = FieldType.Date,
    format = {},
    pattern = "uuuu-MM-dd'T'HH:mm:ss.SSSSS'Z' || uuuu-MM-dd'T'HH:mm:ss.SSS'Z' || uuuu-MM-dd'T'HH:mm:ss.SSS || "
)
private LocalDateTime createdAt;
```

👍 🍷 1

저는 해당하는 pattern 형식을 모두 기입해주는 방식으로 임시방편을 만들고 구현했었는데, Instant 형식으로 선언하고 time 라이브러리를 통해서 맞춰주는 방식도 정말 좋은 것 같습니다.

입력값에 구애받지않고 일관된 데이터를 반환해 줄 수 있을 것 같아요!

Swagger UI 활용 상세한 API 명세 작성

ProductPost Search

elasticSearch를 통한 다양한 상품 게시물 목록 조회 API

GET

/api/product-posts/search

상품 게시물 통합 검색

Elasticsearch를 활용한 상품 게시물 검색 API입니다.

주요 기능

- 전문 검색: 검색어(q)를 통한 제목, 상품명, 설명 전문 검색
- 다중 필터링: 카테고리, 가격 범위, 태그, 상품 상태, 거래 상태 조합 검색
- 페이징: page, size 파라미터 지원
- 다양한 정렬: 관련도순, 인기순, 가격순, 최신순 지원

검색 예시

- 검색어만: ?q=아이폰
- 검색어 + 필터: ?q=맥북&category=전자기기&minPrice=1000000&maxPrice=2000000
- 필터만: ?category=의류&status=NEW&sort=newest
- 태그 검색: ?tags=전원경,중고&sort=popular

Responses

Code

Description

200

검색 성공

Media type

application/json

Examples

검색어 기반 전체 결과

Controls Accept Header

Example Value

```
{
  "pageable": {
    "totalPages": 1,
    "currentPage": 1,
    "pageSize": 10,
    "pageNumber": 1,
    "sort": {
      "property": "id",
      "direction": "DESC"
    },
    "offset": 0,
    "last": true
  },
  "totalElements": 1,
  "totalPages": 1,
  "currentPage": 1,
  "pageSize": 10,
  "pageNumber": 1,
  "sort": {
    "property": "id",
    "direction": "DESC"
  },
  "offset": 0,
  "last": true
}
```

Example Description

검색어 '아이폰'으로 검색한 결과

400

잘못된 요청

Media type

application/json

Examples

무효한 정렬 상태

Controls Accept Header

Example Value

Schema

```
{
  "code": "ERR",
  "message": "입력값 오류에 의해 발생했습니다.",
  "errors": [
    {
      "field": "sortBy",
      "message": "잘못된 정렬 상태입니다."
    }
  ]
}
```

tags

string

(query)

태그 (콤마로 구분)

Example: 미개봉,무료배송

미개봉,무료배송

status

string

(query)

상품 상태

NEW: 새 상품

GOOD: 중고 상품

ALL: 전체 (default)

Default value: ALL

ALL

tradeStatus

string

(query)

거래 상태

SELLING: 판매중

ALL: 전체 (default)

Default value: ALL

ALL

sort

string

(query)

정렬 기준

score: 관련도 순 (default)

popular: 인기순 (좋아요+조회수)

price\_asc: 가격 낮은 순

price\_desc: 가격 높은 순

newest: 최신 등록순

Default value: score

score

- 질문과 요청에 적극적으로 응답하고, 모르는 부분은 화면 공유와 예시 코드로 자세히 설명했습니다.
- PR 리뷰 시 문제 원인과 개선 방향을 구체적으로 제시해 팀원이 더 나은 방식으로 구현할 수 있도록 도왔습니다.
- API 명세서, 회의록 등 체계적인 문서화를 통해 팀원과 프론트엔드가 동일한 기준으로 협업할 수 있도록 노력했습니다.

» 명확한 소통과 체계적인 정리 역량을 바탕으로, 협업 비용을 줄이고 팀의 개발 효율을 높이는 데 기여하겠습니다.

# 중고 거래 플랫폼 연근마켓

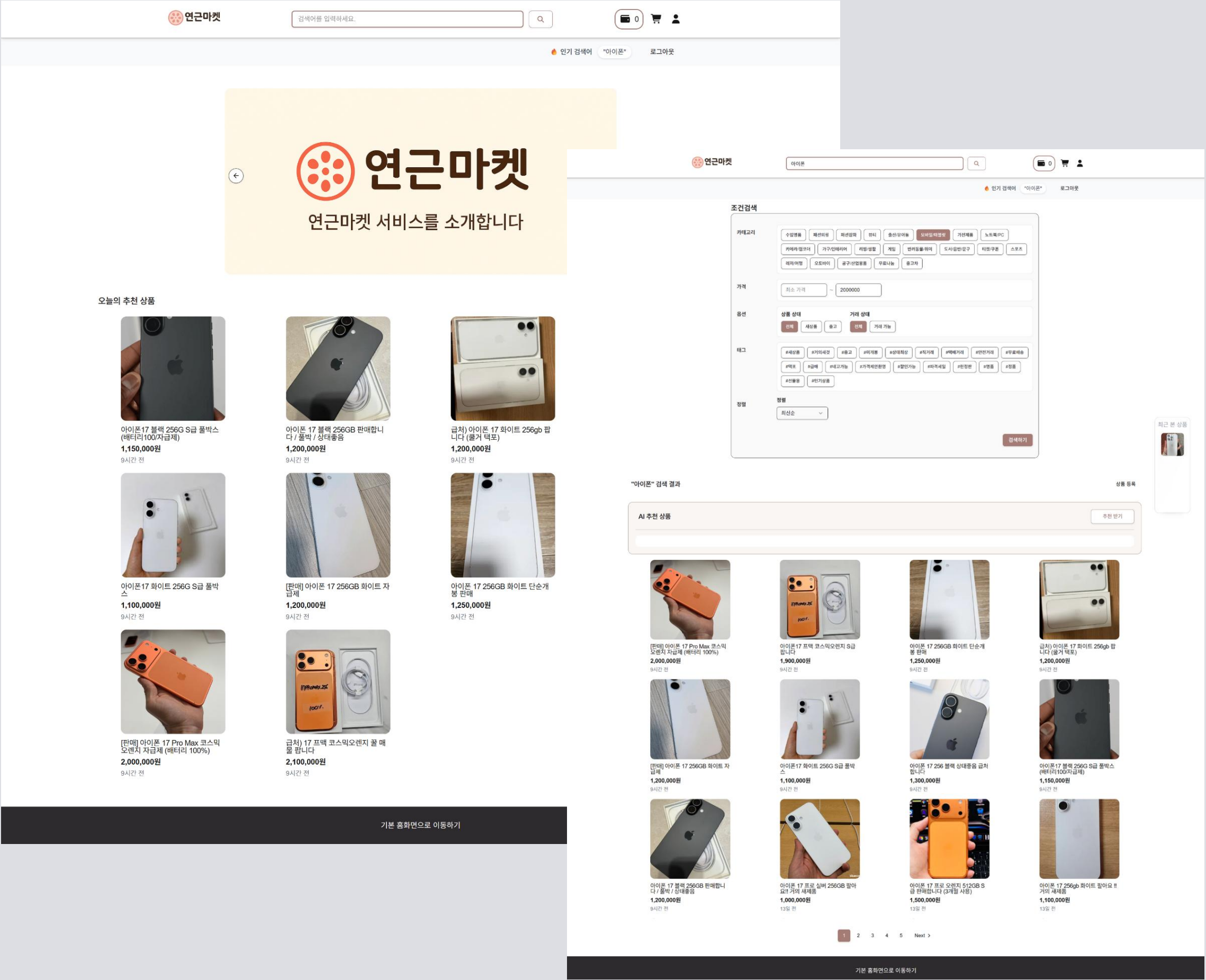
기간 2025.11 ~ 2025.12 (6주)

인원 총 5명 ( BE 5 )

역할 상품 도메인(Elasticsearch 검색/추천 기능 구현),  
서버 오류 알림 시스템 구축

기술 Java, Spring Boot(JPA), Spring Cloud Eureka, Kafka  
MySQL, **Redis**, **Elasticsearch**  
AWS EC2, GitHub Actions, **Sentry**  
GitHub, Slack, **Swagger UI**

링크 [https://GitHub.com/okjunghyeon/beadv1\\_1\\_ctrlz\\_BE](https://GitHub.com/okjunghyeon/beadv1_1_ctrlz_BE)



# 중고 거래 플랫폼 연근마켓

## 프로젝트 소개

Spring Cloud MSA 기반 중고 거래 플랫폼으로,  
Elasticsearch 검색 엔진과 Kafka 이벤트 아키텍처를 활용한 실무형 거래 시스템 개발

## 문제 정의

- RDB 기반 상품 검색의 속도 및 정확도 한계
- 단순 상품 목록 조회 방식으로 사용자 맞춤 추천 기능 부재
- 모놀리식 아키텍처의 장애 전파 및 서비스 확장성 부족
- 중고 거래 결제/정산 과정에서 발생하는 중복 처리 및 데이터 정합성 문제

## Tasks

- Elasticsearch 도입으로 고성능 통합 검색 및 상품 추천 시스템 구현
- Spring Cloud MSA로 6개 서비스 분리, 독립적 운영 환경 구축 및 장애 전파 최소화
- Kafka를 통한 서비스 간 비동기 통신으로 주문/결제/정산 흐름 처리
- 예치금 시스템 구현 및 외부 결제 API 연동을 통한 안전한 거래 환경 구축

## 주요 역할

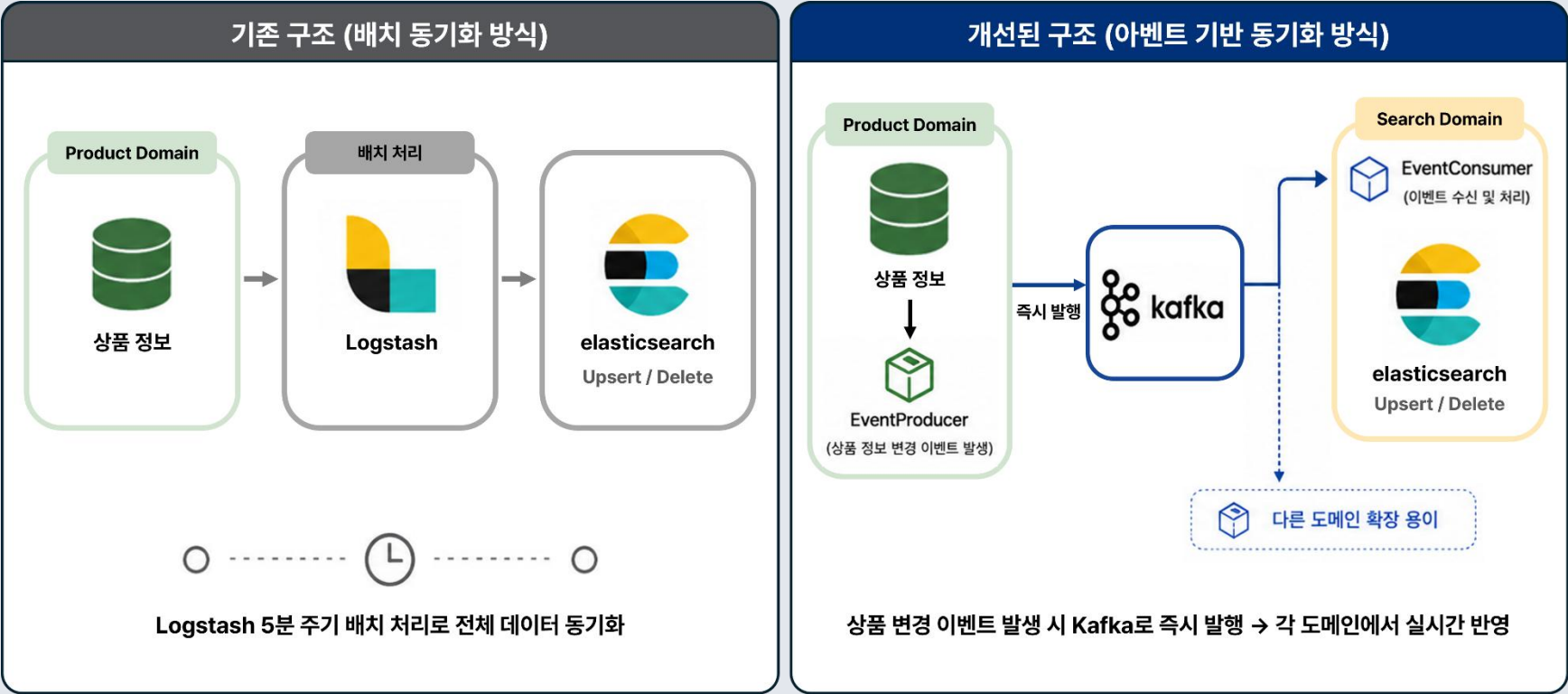
- 상품 CRUD 및 거래 상태 관리 API 구현
- **Redis** Sorted Set(ZSet) 기반 **최근 본 상품** 및 **관심 상품** 관리
- **Elasticsearch** 복합 필터 및 Nori 형태소 분석기 활용 **통합 검색 구현**
- 사용자 경험 개선을 위한 유사 상품 / 인기 상품 추천 기능 구현
- **Swagger** 문서 자동화 및 Sentry-Slack 연동 **에러 모니터링 시스템** 구축

## 성과 및 느낀점

- **Elasticsearch** 도입으로 RDB 대비 **검색 응답 속도 개선**  
→ 성능 문제 해결을 위한 적절한 기술 선택의 중요성 인식
- **다양한 상품 추천 기능 구현**으로 사용자 체류 시간 증가 유도  
→ 기술적 구현이 사용자 경험과 비즈니스 지표에 직결될 수 있음을 인식
- MSA 환경에서 **Swagger 문서 자동화** 및 성공/실패 케이스 명세화  
→ 실제 협업 시 발생하는 커뮤니케이션 비용 최소화를 고려한 개발 습관 체득
- **Sentry-Slack 연동**으로 서버 오류 인지 및 초기 대응 시간 1시간 → 10분 단축  
→ 개발뿐만 아니라 예외 상황에 대한 빠른 인지와 대처의 중요성 인식
- MSA 환경에서 서비스 간 독립적 배포 및 장애 전파 최소화 경험  
→ 대규모 서비스 구조에서의 도메인 분리 설계의 중요성 체감

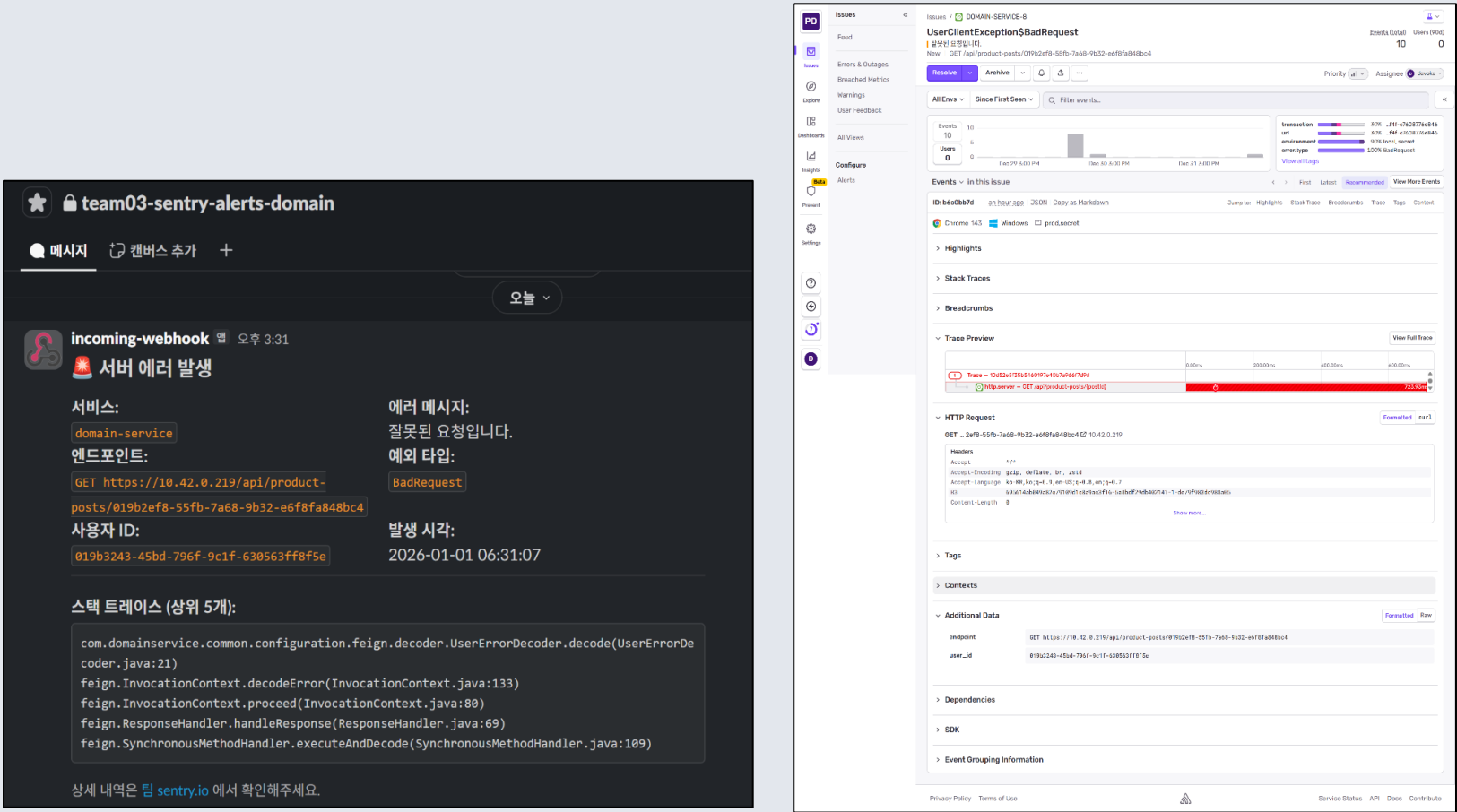
# Spring Cloud MSA + Elasticsearch 기반 중고 거래 플랫폼 연근마켓

## RDB → Elasticsearch 동기화 구조 개선



- 개발 초기 Logstash를 통한 전체 상품 데이터 동기화 방식 선택 (5분 주기 배치 처리)
- 상품 데이터 증가(10만 건↑) 시 동기화 지연으로 검색 결과와 실제 상품 정보 불일치 문제 발생
- 상품 정보 변경 시 Kafka 이벤트 발행, 변경 분만 Elasticsearch 반영하도록 구조 개선
- 동기화 시간 99% 단축 (5분 → 3초) 및 데이터 정합성 문제 해결,  
서버에 Logstash 제거로 서버 메모리 2GB 절감
- Consumer 추가만으로 타 도메인에서 실시간 상품 데이터 활용 가능해짐

## 배포 서버 에러 대응에 대한 고민



- 서버 오류 발생 시 Slack으로 요약된 알림 수신
- 운영 환경 서버 에러 발생 시 대응 프로세스 부재로 평균 30분~1시간의 대응 지연 발생
- Slack Webhook 연동으로 요약된 에러 정보를 팀 채널에 즉시 전송, 전체 팀원 실시간 인지
- 에러 모니터링 툴 Sentry로 상세 스택 트레이스 확인 및 원인 분석, 신속한 대처 방안 강구
- 대응 시간 83% 단축 (1시간 → 10분), 장애 지속 시간 최소화로 사용자 경험 개선 유도

# 대학교 통합 관리 서비스 UniHub

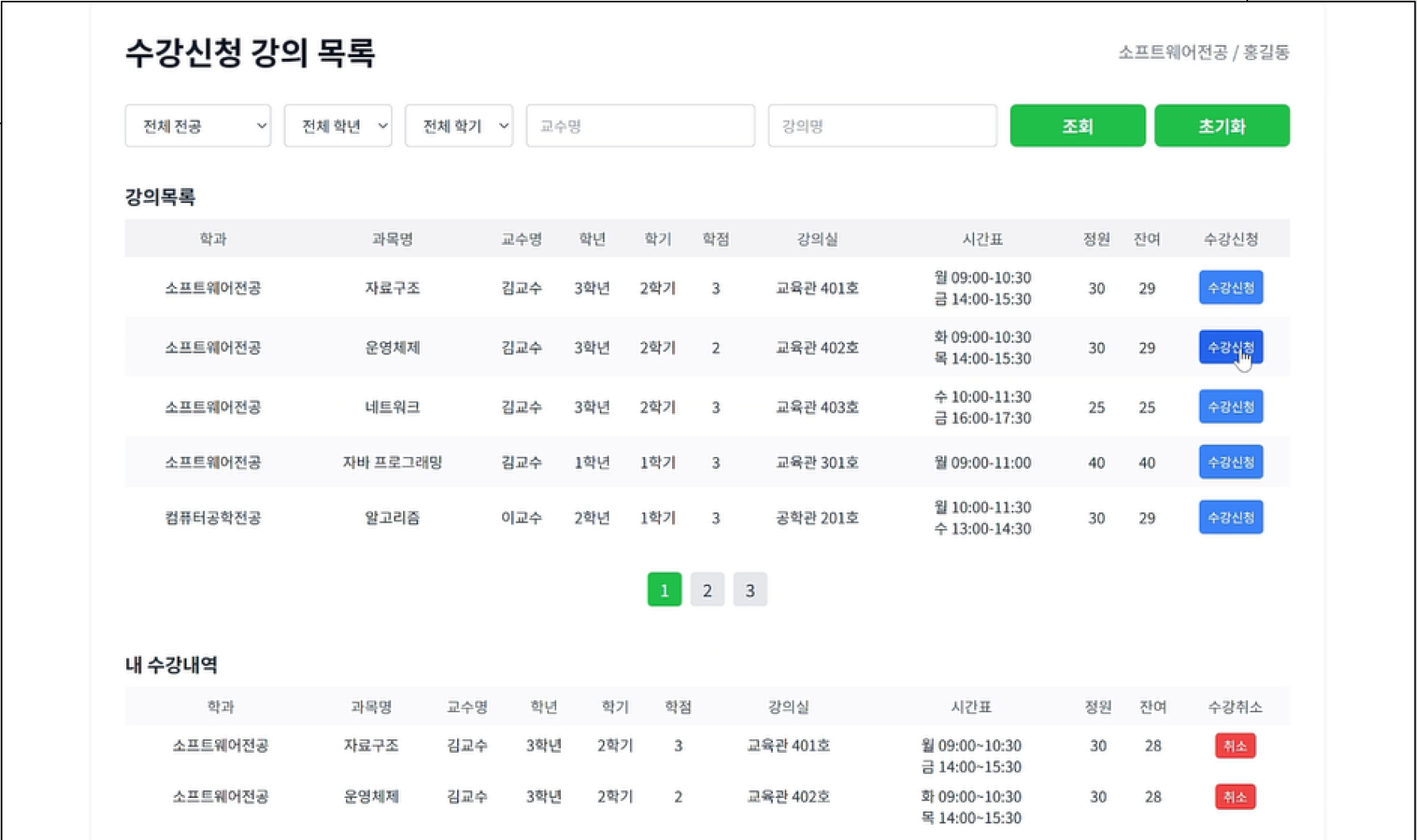
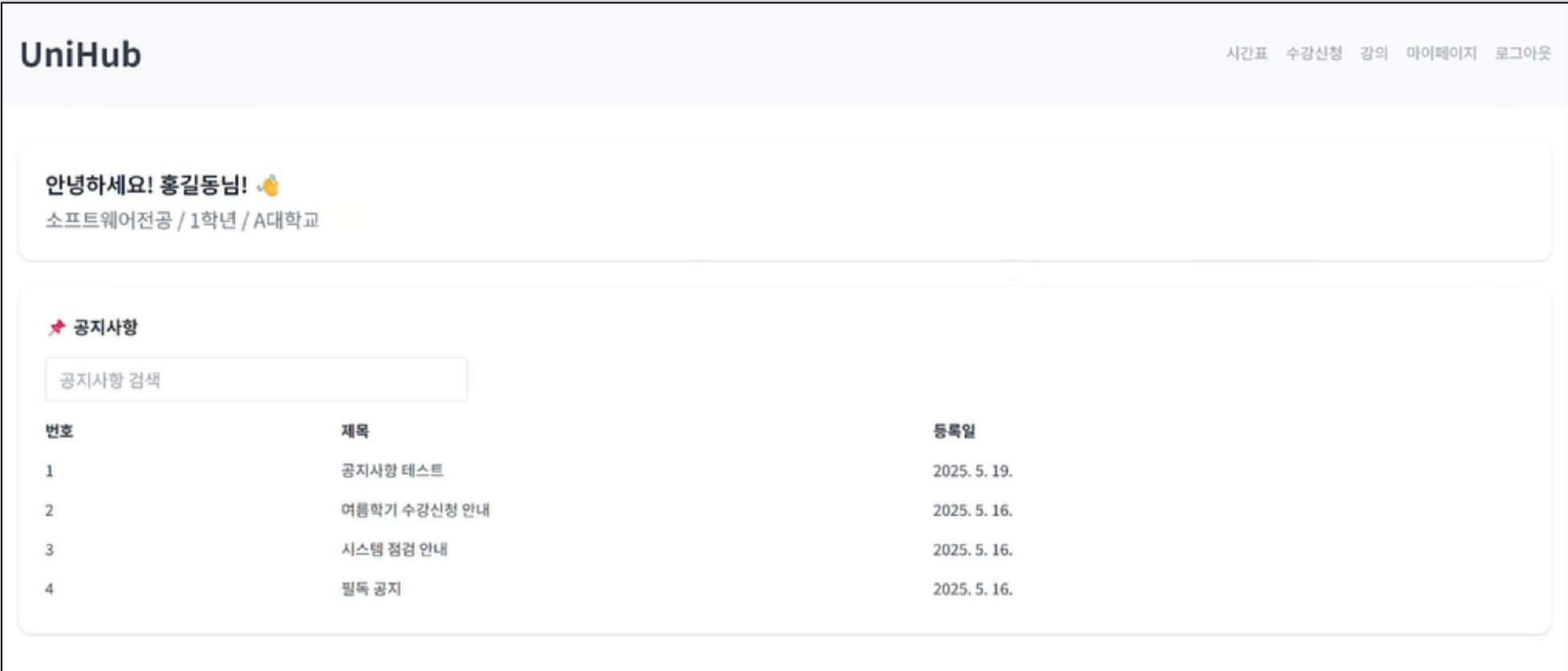
**기간** 2025.05 ~ 2025.06 (6주)

**인원** 총 6명 ( BE 5, FE 1 )

**역할** 수강 신청 도메인 개발, AWS EC2 기반 CI/CD 구축

**기술** Java, Spring Boot(JPA), MySQL, **Redis (Redisson)**  
**AWS EC2, GitHub Actions**, Terraform,  
**Docker**, HAProxy, Nginx  
GitHub, Slack, Notion, Swagger UI, K6

**링크** [https://GitHub.com/okjunghyeon/WEB4\\_5\\_GPT\\_BE](https://GitHub.com/okjunghyeon/WEB4_5_GPT_BE)



## 대학교 통합 관리 서비스 UniHub

## 프로젝트 소개

수강 신청, 시간표, 강의 목록 등 편의 기능을 하나의 서비스로 통합한 대학교 웹 플랫폼으로, 짧은 시간에 요청이 몰리는 수강 신청의 정합성 확보와 안정적인 운영 환경 구축을 목표로 개발

## 문제 정의

- 수강 신청 시 동시 요청에 따른 정원 초과 및 중복 신청 방지 필요
- 최대 학점, 시간표 충돌 등 수강 신청 시 예외 검증 필요
- 대량의 트래픽으로 인한 DB 트랜잭션 부하와 응답 지연 문제에 대한 해결
- 테스트, 운영 환경 분리와 배포 자동화 필요

## Tasks

- 수강 신청, 시간표, 관리자 기능 등 대학 통합 서비스 구현
- 수강 신청 동시 요청에 대한 처리 구조 설계 및 정합성 보장
- AWS 인프라 구성 및 CI/CD 파이프라인 구축
- 테스트 서버와 운영 서버를 분리한 배포 및 예외 발생 시 검증 프로세스 구축
- 문서화, 배포 자동화를 통한 협업 환경 개선 및 팀 개발 생산성 향상

## 주요 역할

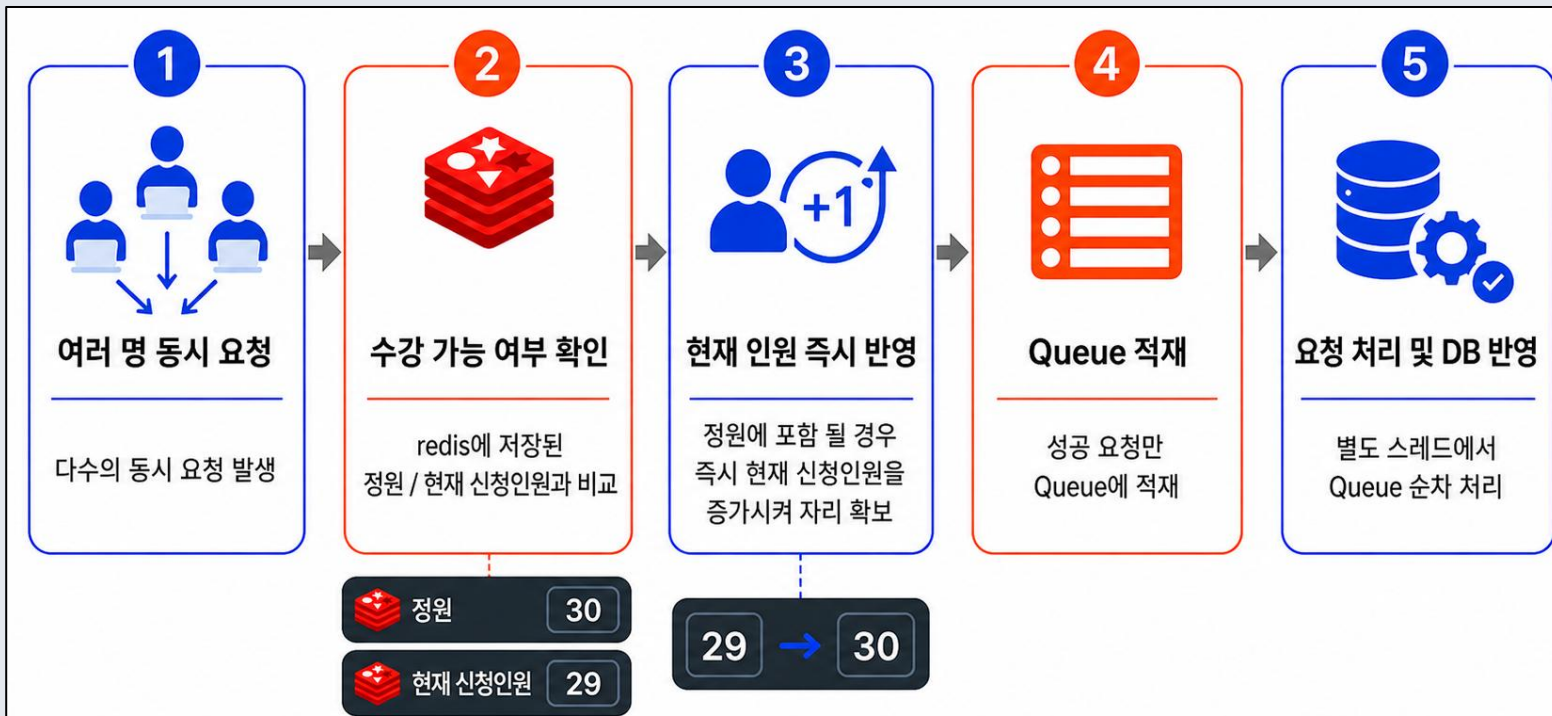
- **수강 신청 도메인 핵심 비즈니스 로직 설계 및 구현**
  - 1) **Redis AtomicLong** 기반 **수강 신청 인원 제어** 로직 설계 및 구현
  - 2) 수강 신청 순차 처리를 위한 **큐 기반 비동기 처리** 구조 설계 및 구현
  - 3) **분산락을 통한 동일 사용자 중복 요청 방지** 로직 설계 및 구현
- **Docker, GitHub Actions 기반 AWS 인프라 구성 및 CI/CD 파이프라인 구축**
- **테스트용 서버 추가 구축** 및 프론트엔드 개발자와 연동 전 예외 검증 프로세스 구축
- 로드밸런서(HAProxy) 기반 무중단 배포 적용

## 성과 및 느낀점

- **정원 30명 강의에 1,000명 동시 요청 상황에서도 정합성 확보**
  - 동시성 문제는 단순 차단이 아닌 요청의 특성에 맞는 처리 구조 설계가 중요함을 체감
- **Redis 기반 분산락을 적용해 동일 사용자 반복 요청을 1회로 제한하고 중복 신청 방지**
  - 동시성 제어는 단순 성능 문제가 아닌 사용자 단위 요청 제어와 예외 상황을 고려해야 함을 인식
- **Redis 원자 연산과 큐 분리를 통해 DB 트랜잭션 부하 완화 및 순차처리 보장**
  - 빠른 응답과 안정적 처리를 함께 고려하는 단계별 역할 분리 설계의 중요성 학습
- **테스트 서버 운영을 통해 배포 전 검증 후 운영 서버에 반영하는 프로세스 구축**
  - 예상치 못한 오류로 인한 협업 비용을 줄이고, 빠른 복구와 안정적인 운영이 중요함을 인식
- **HAProxy 기반 Blue/Green 배포로 중단 없이 신규 버전을 반영하는 무중단 배포 체계 확보**
  - 기능 구현만큼이나 배포 과정 자체를 설계하는 것도 안정적인 운영의 핵심이라는 점을 체감

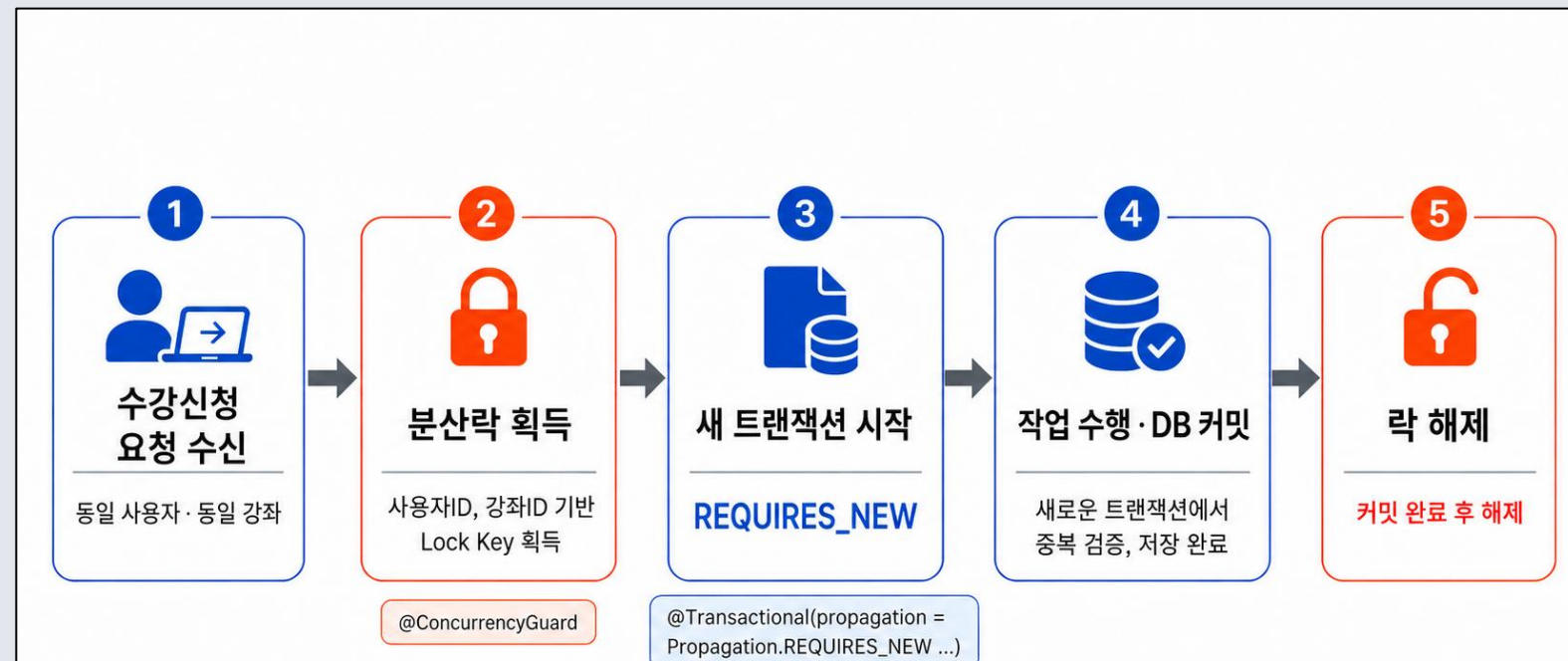
## 대학교 통합 관리 서비스 UniHub

## Redis(Redisson) 기반 수강 신청 정원 제어 구조 설계

정원 제어와 DB 저장을 분리해 **정합성**과 응답 속도 확보

- 개발 초기에는 수강 신청 시 동시 DB 접근을 막기 위해 강좌 단위 분산락을 고려  
→ 해당 방식은 한 명의 트랜잭션이 끝날 때까지 다른 요청이 모두 대기해 병목이 생기는 문제가 발생
- 이를 해결하고자 **정원 제어와 실제 DB 저장 과정을 분리**하는 방향으로 처리 구조를 재설계
- Redis에서 현재 신청 인원을 먼저 반영**하고, 정원 내 요청만 별도 스레드에서 순차 처리하도록 구성
- 결과적으로 **DB 트랜잭션 폭주를 막고 정합성과 응답 속도를 함께 확보**함

## 분산락 + REQUIRES\_NEW를 활용한 중복 요청 방지

중복 요청을 방지하고, 커밋 완료 후 **락 해제**로 정합성 보장

- 동일 사용자의 같은 강좌 반복 요청 시 중복 신청 및 중복 작업 삽입 위험이 발생
- 이를 해결하고자 **동일 사용자 · 동일 강좌 요청에 분산락을 적용**하여 동시 진입 제한
- 단, 분산락만 적용한 경우 **DB 커밋 시점보다 락 해제 시점이 빨라** 요청이 정상 반영되지 못하는 문제 발생
- REQUIRES\_NEW 적용**으로 락 획득 시 **새 트랜잭션을 열어 작업 · 커밋을 완료**하는 방식으로 구조 보완
- 결과적으로 **반복 요청 1회 제한** 및 **수강 신청 정합성을 확보**할 수 있었음



# THANK YOU

감사합니다

옥정현

devoks@naver.com